How to edit the catalogue

Everything lives in **`catalogue.json`**. You edit that one file, run two commands, and the web page, the Word document and the CSV all update together. You never edit the `.html` or the `.docx` directly — they get overwritten.

```
catalogue.json  ──▶  python3 build_web.py   ──▶  .csv + .html
                └──▶  node build_docx.js    ──▶  .docx
```

Three rules

1. **Text goes in double quotes.** `"access": "Open"` — never single quotes.
2. **Every line ends with a comma, except the last one in a block.** This is the mistake that breaks the file 90% of the time.
3. **Don't invent field names.** Every dataset uses the same 30 keys: the 28 catalogue fields that become CSV columns, plus `spatial_grain` and `temporal_grain`, which are whole numbers rather than text. If a field doesn't apply, write `"Not applicable"` — don't delete the line, or the table will have a hole in it.

To check you haven't broken anything before rebuilding:

```
python3 -c "import json; json.load(open('catalogue.json')); print('valid')"
```

If it prints an error with a line number, go to that line — the problem is almost always a missing or extra comma just above it.

That check only proves the file is readable JSON. `build_web.py` then checks that it is a usable catalogue: every record has all 30 keys, every ID is unique and shaped like `HEA-001`, every `domain` is one that `catalogue.domains` declares, every entry in `projects` is a declared project code, and both grain ranks are whole numbers from 1 to 5. It reports *every* problem it finds in one go, each naming the record and the field, and refuses to write anything until they are fixed. So the fastest way to check your edit is simply to run the build.

Recipe A — change a field on an existing dataset

Find the record by its ID, change the text, save.

Example: dew-point was added to the Pará download (fix 1). Search for `"CLI-001"`, then edit:

```
"limitations": "Temperatures arrive in Kelvin (subtract 273.15). ... The Pará download was extended to dew-point temperature in August 2026; series before that date do not contain it."
```

That's the whole operation. The web page and the Word file pick it up on the next build.

Which field to touch, for the fixes you sent:

Your fix	Record	Fields changed
1 · humidity for Pará	`CLI-001`, `DER-003`	`limitations`, `key_variables`, `description`
3 · human footprint source	`ENV-007`	nearly all — see Recipe C
4 · COVISA after ethics	`HEA-003`	`sharing_conditions`, `holding_status`, `record_status`, `key_variables`
5 · CGE at project start	`CLI-004`	`sharing_conditions`, `holding_status`, `record_status`
6 · INMET bias check	`CLI-005`	`description`, `holding_status`, `limitations`

Recipe B — add a new dataset

Copy any existing record, paste it below, change the values. Two things are easy to forget:

1. Give it an ID that doesn't exist yet. The prefix sets the colour and the section it lands in: `HEA` health, `CLI` climate, `ENV` environment, `REF` reference geography, `DER` derived by you.

2. Set `spatial_grain` and `temporal_grain`. These drive the two-bar resolution mark on the web page. Whole numbers from 1 (coarse) to 5 (fine). They sit in the record itself, each one directly below the field it ranks, so there is no second file to remember.

```
"native_resolution": "Gauge point, daily",
"spatial_grain": 4,
...
"temporal_resolution": "Daily",
"temporal_grain": 5,
```

Rough scale, if you're unsure:

	1	2	3	4	5
Space	global index	~9 km	~1 km, municipality	100–250 m	30 m or finer, point, address
Time	static	annual	monthly	weekly	daily or hourly

Miss this step and the build stops and tells you which record is missing which rank. It used to default quietly to 3/3, which meant a wrong resolution mark could reach the page unnoticed.

Recipe C — the blank template

Copy this into the `datasets` list and fill it in.

```
{
  "id": "CLI-007",
  "title": "",
  "title_pt": "",
  "domain": "Climate",
  "projects": ["ACL-AMZ"],
  "description": "",
  "key_variables": "",
  "custodian": "",
  "producer": "",
  "contact": "",
  "spatial_coverage": "",
  "native_resolution": "",
  "spatial_grain": 3,
  "analysis_unit": "",
  "temporal_coverage": "",
  "temporal_resolution": "",
  "temporal_grain": 3,
  "update_frequency": "",
  "latency": "",
  "access": "Open",
  "access_url": "",
  "formats": "",
  "licence": "",
  "sharing_conditions": "",
  "personal_data": "No",
  "linkage_key": "",
  "limitations": "",
  "holding_status": "Candidate - not yet acquired",
  "steward": "IPSP Climate and Health Laboratory",
  "record_status": "Draft"
}
```

`steward` names whoever is answerable for *this record* — if you are contributing on behalf of another Pasteur Network member, put your own institute and team there, not the example above. The same goes for `custodian`, `producer` and `contact`: they describe who holds and who produced the data, and they are right only if they are literally true.

Three fields have a fixed set of values, because the web page filters and colours on them:

- `domain` — `Health` · `Climate` · `Environment` · `Reference` · `Derived`. To introduce a sixth, add one line to `catalogue.domains` at the top of `catalogue.json` giving its name and a hex colour; the filter button, the colour and the DOCX section follow from that one line, with no change to either build script.
- `access` — anything starting `Open` is green, `Partly open` is amber, `Restricted` or `Internal` is red
- `projects` — `YF-TMIN` · `DENGUE-SUHI` · `ACL-AMZ`, one or more

Recipe D — update a gap

Gaps live in `gaps.json`, which sits beside the checkout and is not committed. Each one now carries a status, an owner and a date.

```
{
  "gap": "What is missing, in a short phrase.",
  "status": "Planned",
  "detail": "What is missing, in one or two sentences.",
  "action": "The step that closes it.",
  "owner": "IPSP",
  "when": "Next data cycle"
}
```

`owner` names the institute answerable for closing the gap. `status` must be exactly `Resolved`, `In progress` or `Planned` — those three strings set the colour and the label. When something is resolved, don't delete the gap: change the status and rewrite `action` to say what was done. The reviewer wants to see the closed ones, and the label above `action` flips from "Next step" to "What was done" on its own.

Rebuild

```
python3 build_web.py     # → CSV and HTML (validates first, and stops on any problem)
node    build_docx.js    # → DOCX          (on a fresh clone, run `npm install` once)
```

Then bump `"version"` at the top of the file and add a row to the change-log table in `build_docx.js`.

What changed in v0.2

Seven gaps were reviewed and the catalogue went from 24 to 26 datasets. The gap-by-gap detail is internal review material: it lives in `gaps.json`, which is not committed, and is rendered into the DOCX.